

```

#include <iostream>
#include <climits>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = (i == j) ? 0 : INF;
    cout << "Enter edges (Source Destination Weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == INF) {
            adj[u - 1][v - 1] = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

void printAdjMatrix(int adj[MAX][MAX], int n) {
    cout << "\nAdjacency Matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j] == INF) cout << "INF ";
            else cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}

// Dijkstra's Algorithm - single source shortest path, non-negative weights
void dijkstra(int adj[MAX][MAX], int n, int source) {
    int D[MAX]; // shortest distance from source
    bool V[MAX]; // visited flag

    for (int i = 0; i < n; i++) {
        D[i] = INF;
        V[i] = false;
    }
    D[source] = 0;

```

```

while (true) {
    // find minimum distance unvisited node
    int minVal = INF, SN = -1;
    for (int i = 0; i < n; i++) {
        if (!V[i] && D[i] < minVal) {
            minVal = D[i];
            SN = i;
        }
    }
    if (SN == -1) break; // all reachable nodes processed
    V[SN] = true;

    // relax edges from SN
    for (int i = 0; i < n; i++) {
        if (!V[i] && adj[SN][i] != INF &&
            D[SN] + adj[SN][i] < D[i]) {
            D[i] = D[SN] + adj[SN][i];
        }
    }
}

cout << "\nShortest Distance from Node " << source+1 << ":\n";
for (int i = 0; i < n; i++) {
    if (D[i] == INF)
        cout << source+1 << " -> " << i+1 << " = INF\n";
    else
        cout << source+1 << " -> " << i+1 << " = " << D[i] << endl;
}
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    printAdjMatrix(adj, n);
    int source;
    cout << "\nEnter source node: ";
    cin >> source;
    dijkstra(adj, n, source - 1);
    return 0;
}

```

WHEN TO USE DIJKSTRA'S (Scenario Triggers)

- ▶ Question asks for SHORTEST path from ONE source node to all others (or one target)
- ▶ Graph has WEIGHTED edges, and ALL weights are non-negative
- ▶ "Cheapest route", "minimum cost/time/distance to travel from X to Y" style scenarios
- ▶ GPS/road network, flight cost, network latency minimization scenarios
- ▶ Single starting point given — if question needs shortest path between ALL pairs, use Floyd-Warshall instead

Dijkstra's vs Bellman-Ford vs BFS — Which One?

- ▶ Weighted graph, NO negative edges, single source? → Dijkstra's
- ▶ Weighted graph WITH negative edges (or need cycle detection)? → Bellman-Ford instead
- ▶ UNweighted graph, single source shortest path? → plain BFS is simpler and faster
- ▶ Need shortest path between ALL pairs of nodes? → Floyd-Warshall instead
- ▶ Dijkstra's is essentially 'BFS + always pick the cheapest unvisited node next' instead of FIFO order

MANUAL TRACE METHOD (Selected-Node Table, from Class Notes)

Same table style as Prim's, but tracking DISTANCE FROM SOURCE, not edge weight:

1. Make a table: rows = selected nodes so far, columns = ALL nodes in the graph.
2. Start: source row has $D[\text{source}] = 0$, all others = $\text{INF} (\alpha)$.
3. Pick the SMALLEST unvisited distance value in the current row → next selected node.
4. New row: copy previous row, UPDATE any column where $D[\text{selected}] + \text{edge_weight} < \text{current value}$.
5. Circle the value you just picked (this locks in the shortest distance to that node).
6. Repeat until all reachable nodes are selected/circled.

Key difference from Prim's table: you ADD cumulative distance ($D[u] + \text{weight}$), not just compare the raw edge weight alone.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

NEGATIVE weights break Dijkstra's — once a node is marked visited, its distance is assumed FINAL. Negative edges can violate this assumption, giving wrong answers silently.

Dijkstra's finds shortest PATH DISTANCE, not minimum edge weight — don't confuse $D[]$ (cumulative) with Prim's $\text{key}[]$ (single edge).

Directed graphs are fine — unlike Prim's/Kruskal's, Dijkstra's does NOT require an undirected graph.

1-indexed input, 0-indexed storage — same off-by-one risk as all other algorithms here.

If a node stays at INF , it's unreachable from the source — state this explicitly if asked.

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (directed, weighted): $A \rightarrow B=10$, $A \rightarrow C=3$, $C \rightarrow B=1$, $B \rightarrow D=2$, $C \rightarrow D=8$, $D \rightarrow E=7$

Source: A

Step-by-step trace:

- ▶ $D[A]=0$, $D[B]=\text{INF}$, $D[C]=\text{INF}$, $D[D]=\text{INF}$, $D[E]=\text{INF}$
- ▶ Select A ($D=0$) → relax: $D[B]=\min(\text{INF}, 0+10)=10$, $D[C]=\min(\text{INF}, 0+3)=3$
- ▶ Select C ($D=3$, smallest) → relax: $D[B]=\min(10, 3+1)=4$, $D[D]=\min(\text{INF}, 3+8)=11$
- ▶ Select B ($D=4$, smallest) → relax: $D[D]=\min(11, 4+2)=6$
- ▶ Select D ($D=6$, smallest) → relax: $D[E]=\min(\text{INF}, 6+7)=13$
- ▶ Select E ($D=13$) → done, all nodes selected

Shortest distances from A: $A=0$, $B=4$, $C=3$, $D=6$, $E=13$

EXAMPLE 2 — Unreachable Node Scenario

Graph: $A \rightarrow B=5$, $C \rightarrow D=2$ (C, D form a separate component, unreachable from A)

Source: A → $D[A]=0$, $D[B]=5$. $D[C]$ and $D[D]$ remain INF forever (never updated).

Exam expects you to say: "Nodes C and D are unreachable from source A — no directed path exists. Their shortest distance is undefined (INF)."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A delivery company wants the cheapest route from their warehouse to every customer location, given road distances between intersections."

How to apply Dijkstra's: warehouse = source node, intersections = nodes, road distances = non-negative weighted edges. Run Dijkstra's from the warehouse — the resulting $D[]$ array directly gives the cheapest route distance to every customer location simultaneously (single-source, multiple destinations).

EXAMPLE 4 — Negative Weight Trap (Medium/Hard style)

Scenario: exam gives a graph WITH a negative edge and asks you to run Dijkstra's on it directly.

Correct handling: DO NOT just run the algorithm blindly. First state: "Dijkstra's algorithm assumes non-negative edge weights. Since this graph contains a negative edge, Dijkstra's may produce an INCORRECT result because once a node is marked visited, we assume its distance is final — a later negative edge could actually offer a shorter path we've already dismissed. Bellman-Ford should be used instead." This reasoning is often worth more marks than blindly computing a (possibly wrong) answer.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Dijkstra's answer, check:

Did I confirm the graph has NO negative weights before applying Dijkstra's?

Did I pick the SMALLEST cumulative distance at each step, not smallest node index?

Did I relax ALL neighbours of the newly selected node, not just some?

If a node is unreachable, did I explicitly state its distance is INF/undefined?

Did I double check directed vs undirected — did I only set $\text{adj}[u][v]$, matching the graph's actual direction?